# Group Project Proposal

*Andrei Krivoruchko // 22328383*
*Maxim Koloskov // 22348633*
*Sean Keane // 22357956*
*Olaf Madynski // 22371636*

Team Name : 4 Horsemen
Git Repo : https://gitlab.com/ucd-cs-rem/comp30220-2026/4-horsemen

## Problem Statement

In the real world food delivery is a problem that can be viewed as highly distributed and naturally requires a microservices architecture.
A customer placing an order involves multiple independent actions such as (in no particular order) user authentication, both restaurant and menu management, order processing and delivery notifications. Each of which could be handled by separate independently deployable services.

The system must handle concurrent requests from multiple users and remain operational if a given service just fails and scale under load. All this makes food delivery (such as that of Deliver or Just Eat) an excellent idea for distribution thus justifying the use of microservices, event communication and service discovery.

## Proposed System

We want to build a distributed food delivery platform implemented using microservices. Our system will consist of 4 microservices. Each owned and developed by one team member:

| | |
|---|---|
| Order Service | - Place, retrieve and update orders |
| Restaurant Service | - Manage restaurants and menus |
| User Service | - User registration and authentication |
| Notification Service | - Event based order notifications via ActiveMQ |

## Architecture

We will use 2 distinct communication patterns.
REST (Synchronous) - The Order Service, Restaurant Service, User Service expose RESTful HTTP APIs built with Spring Boot. When a customer places an order, the Order Service will make REST calls to the Restaurant Service to check the restaurant and menu items and to the User Service to verify the customer.

Message Oriented Middleware (Asynchronous): When an order is successfully placed, our Order Service will publish an event to an ActiveMQ message queue. The Notification Service subscribes to this queue and processes the event asynchronously. Sending a confirmation to the customer. This decouples the notification concern from the order placement flow.

Service Discovery: A Netflix style Eureka server will act as the service registry. Each microservice registers itself with Eureka on startup. The Order Service uses Eureka to discover the URLs of the Restaurant and User Services dynamically instead of relying on hardcoding. This enables our services to scale horizontally.

## Technology Stack

-Spring Boot
-Spring Data JPA
-PostgreSQL
-Apache ActiveMQ
-Netflix Eureka
-Resilience4j
-Docker
-GitLab

## Scalability and Fault Tolerance

Scalability will be demonstrated by running multiple instances of the Order Service behind a load balancer configured in docker-compose. Eureka will automatically register all instances and incoming requests will be distributed across them.

We will do fault tolerance in 2 ways. Firstly the Resilience4j circuit breaker will be added to the Order Services calls to the Restaurant and User Services. If either service becomes unavailable then the circuit breaker opens and the Order Service returns a graceful error rather than hanging or crashing.
Second, ActiveMQ provides builtin fault tolerance for the notification flow that is to say, if the Notification Service is temporarily down our messages are held in the queue and delivered whenever it recovers.

## Work Division

Andrei Krivoruchko - Order Service, Docker Compose setup, Eureka client integration, Circuit Breaker, Basic frontend
Olaf Madynski - Restaurant Service, Eureka client integration, menu data model
Maxim Koloskov - User Service, Eureka client integration, user authentication
Sean Keane - Notification Service, ActiveMQ setup, Eureka Server